Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Сервис-ориентированная архитектура*»

Отчёт
По лабораторной работе №2

Вариант: 21

Сдающие:
Петров Вячеслав Маркович
Елисеев Константин Иванович

группа P3408

Преподаватель:
Кривоносов Егор Дмитриевич

г. Санкт-Петербург, 2026 г.

Задание

На основе разработанной в рамках лабораторной работы #1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

- Первый ("вызываемый") веб-сервис должен быть реализован на фреймворке Spring MVC REST и развёрнут в окружении под управлением сервера приложений Tomcat.
- Второй веб-сервис должен быть реализован на фреймворке Spring MVC REST, развёрнут в окружении под управлением ещё одного экземпляра сервера приложений Tomcat и вызывать REST API первого сервиса.
- **Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией!**
- Доступ к обоим сервисам должен быть реализован по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов - включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид - параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности, о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

Выполнение

Исходный код

https://github.com/virdxk/ticket-api-openapi

- https://github.com/virdxk/ticket-api-openapi/tree/codex/nested-field-labels/ticket-service - Ticket Service
- https://github.com/virdxk/ticket-api-openapi/tree/codex/nested-field-labels/person-service - Person Service
- https://github.com/virdxk/ticket-api-openapi/tree/codex/nested-field-labels/booking-service - Booking Service
- https://github.com/virdxk/ticket-api-openapi/tree/codex/nested-field-labels/common - общий модуль: обработка ошибок, разбор фильтров и сортировки, поддержка JSON Merge Patch
- https://github.com/virdxk/ticket-api-openapi/tree/codex/nested-field-labels/client - клиентское приложение
- https://github.com/virdxk/ticket-api-openapi/tree/codex/nested-field-labels/deploy - скрипты развёртывания на helios

Реализация сервисов

Ticket Service, Person Service и Booking Service написаны на Java 17 и Spring Boot 3.5.16 со Spring MVC REST. Maven собирает каждый сервис в WAR-архив. Интерфейсы контроллеров, модели и клиенты Booking Service генерируются из исходных YAML-спецификаций плагином openapi-generator-maven-plugin. Сгенерированный код вручную не изменяется.

Запрос проходит через контроллер, сервис и репозиторий. Сущности JPA описывают таблицы PostgreSQL. Ticket Service и Person Service работают независимо, а Booking Service хранит связь человека с билетом. В common оставлены общие механизмы:

- ApiExceptionHandler возвращает ошибки в формате {message, time, details}. При нарушении ограничений тела запроса ответ 422 содержит только первую ошибку в details
- QueryParameters разбирает фильтры и сортировку, проверяет имена полей и преобразует значения из строк в числа, даты и перечисления

- JsonNullableModule подключает обработку nullable-полей. Это позволяет отличить отсутствующее поле PATCH от явно переданного null

Тексты ошибок находятся в messages_ru.properties и messages_en.properties каждого сервиса. Общие сообщения и переводы ограничений валидации хранятся в common-messages_ru.properties и common-messages_en.properties. Ключи сообщений и строковые константы собраны в utils/Constants.java. По умолчанию используется русский язык. Сам сервис выбирает язык по Accept-Language, но текущий PHP-прокси этот заголовок не передаёт.

JPQL-запросы подсчёта и выборки страницы записаны в сервисах. Фильтрация, сортировка и ограничение числа записей выполняются в базе данных. Нумерация страниц начинается с 1, размер страницы ограничен 100. Параметры filter и sort можно повторять. Фильтры объединяются через AND, противоречивые условия возвращают пустую страницу. Значения передаются в запрос как параметры.

PATCH изменяет только переданные поля. Для nullable-полей price, type и venue явный null очищает значение. Если поле отсутствует, его прежнее значение сохраняется. Пустой PATCH отклоняется с кодом 422. Неизвестные поля JSON и дробные числа вместо целых отклоняются настройками Jackson в application.properties.

Некорректные параметры URL, неразбираемый JSON, неподдерживаемый метод и Content-Type возвращают 400, отсутствующий объект возвращает 404. Остальные HTTP-ошибки сохраняют свой статус, а необработанные исключения возвращают 500. Клиент получает локализованное сообщение без внутренних подробностей исключения.

RemoteClientsConfig настраивает сгенерированные REST-клиенты Booking Service, доверенный сертификат и тайм-ауты. RemoteServices выполняет вызовы и переводит ошибки соседних сервисов в ответы API. Недоступность соседнего сервиса возвращает 503. Продажа VIP-копии создаёт новый билет с удвоенной ценой и бронирование. При отсутствии цены или её переполнении возвращается 409. Если запись бронирования не удалась, сервис пытается удалить созданную копию билета. Отмена бронирований возвращает идентификатор человека и число удалённых записей.

Транзакции заданы на методах, которым нужна атомарная работа с базой. Booking Service выполняет удалённые вызовы до локальной транзакции. Для внедрения зависимостей используются final-поля и конструкторы Lombok RequiredArgsConstructor.

Проверка запросов

В корне проекта подготовлена коллекция ticket-service.postman_collection.json для импорта в Postman. Запросы обращаются к Гелиосу и покрывают создание, получение, изменение и удаление билета, фильтрацию, сортировку и специальное удаление по типу. После создания идентификатор сохраняется для следующих запросов. Отдельные запросы проверяют ошибки 400, 404 и 422. Для coordinates.y проверяются допустимая граница 747 и значение 748, которое должно вернуть 422 с ошибкой этого поля. Для comment подготовлен запрос с 682 символами при максимальной длине 681.

Сборка Maven, сборка Vite и проверка архитектуры Steiger прошли. Автотесты не запускались. После развёртывания списки трёх сервисов вернули 200. Для Ticket Service проверены CRUD, PATCH с null, фильтры и сортировка. Значение y=747 принято. Значение y=748 и комментарий на 682 символа вернули 422 с одной ошибкой поля. Неподдерживаемый метод и Content-Type вернули 400. Английский перевод проверен прямым HTTPS-запросом к Tomcat.

Развёртывание

Каждый сервис развёрнут в отдельном экземпляре Tomcat 10.1.60 на helios.

Публичные запросы идут через se.ifmo.ru. Для каждого сервиса в public_html есть каталог с .htaccess и PHP-прокси, который пересылает запрос в Tomcat по HTTPS и возвращает ответ. Ticket Service использует порт 41390, Person Service порт 41391, Booking Service порт 41392. Клиент лежит в public_html/lab2. Настройка ddl-auto=validate проверяет существующую схему PostgreSQL при запуске и не изменяет её.

Компонент	Публичный адрес
Ticket Service	https://se.ifmo.ru/~s412939/ticket-service/tickets
Person Service	https://se.ifmo.ru/~s412939/person-service/persons
Booking Service	https://se.ifmo.ru/~s412939/booking-service/booking/bookings
Клиент	https://se.ifmo.ru/~s412939/lab2/
Swagger UI	https://se.ifmo.ru/~s412939/

Доступ по https

Самоподписанный сертификат создаёт утилитой keytool скрипт развёртывания при первом запуске на helios: хранилище ключа keystore.p12, хранилище доверенных сертификатов truststore.p12 и сертификат server.crt. Пути к ним и пароли указаны в server.xml Tomcat и в application.properties Booking Service. В server.xml есть только https-коннектор, поэтому запросы по http отклоняются.

```
$ curl http://localhost:41390/tickets
Bad Request
This combination of host and port requires TLS.


$ curl https://localhost:41390/tickets
curl: (60) SSL certificate problem: self-signed certificate

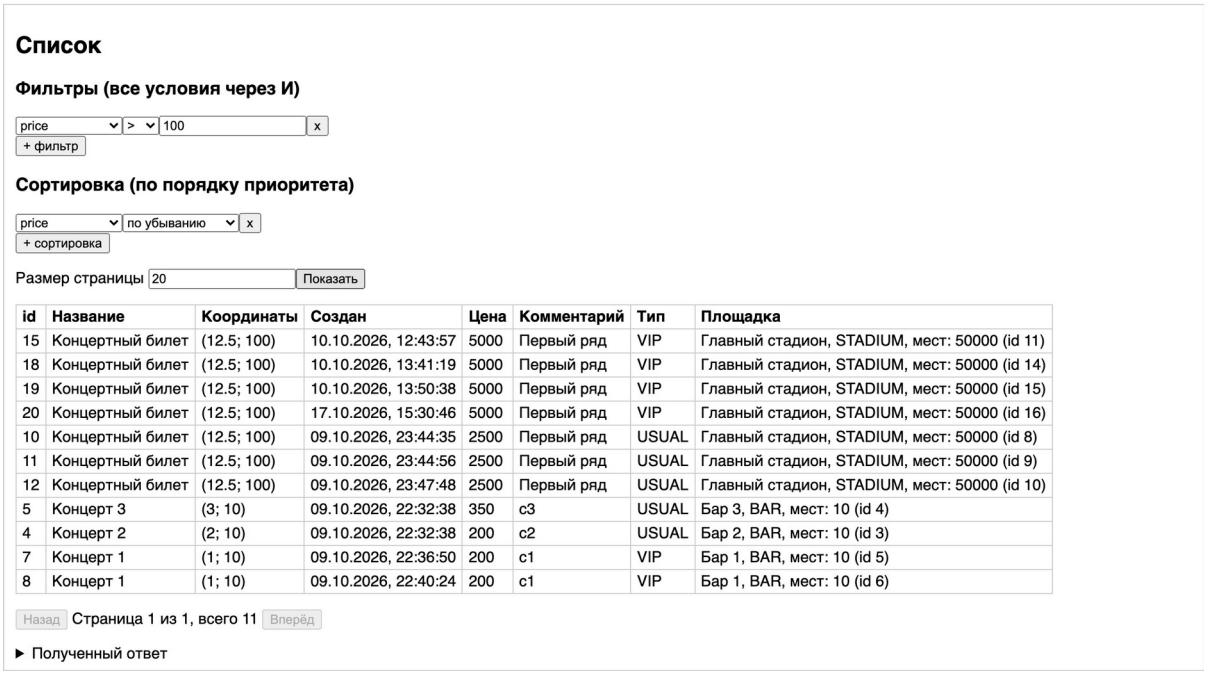

$ curl --cacert ~/public_html/ticket-service/server.crt
https://localhost:41390/tickets/2
{"id":2,"name":"2222","coordinates":{"x":1.0,"y":2.0},"creationDate":"2026-10-
09T19:30:31.184Z","price":null,"comment":"","type":null,"venue":null}
```

Клиентское приложение

https://se.ifmo.ru/~s412939/lab2/

Клиент написан на React и собирается Vite. Для каждой коллекции есть список с фильтрацией, многоуровневой сортировкой и постраничным выводом, получение, создание, частичное обновление и удаление элемента. Отдельно доступны специальные операции. Данные выводятся таблицами, поля и значения перечислений подписаны по-русски. Формы проверяют ввод до отправки. При ошибке сервера показываются код ответа, сообщение и, если он есть, один пункт details с ошибкой поля.

Рисунки 1-3 сохранены из первоначальной проверки сценариев. В текущей версии оформление клиента обновлено.



Список

Фильтры (все условия через И)

price > 100 x

+ фильтр

Сортировка (по порядку приоритета)

price по убыванию x

+ сортировка

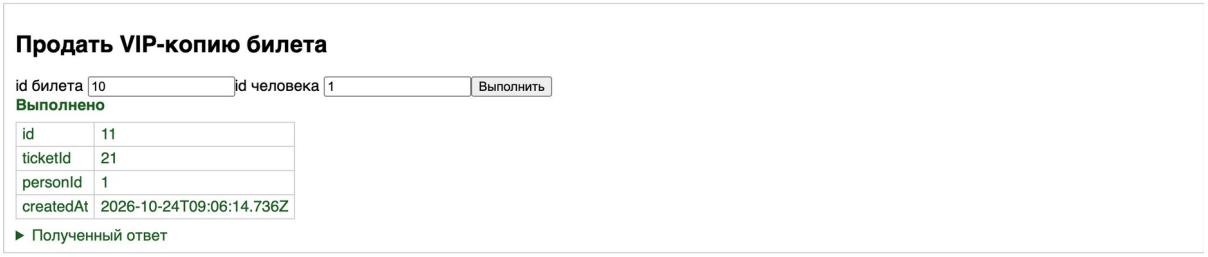
Размер страницы 20 Показать

id	Название	Координаты	Создан	Цена	Комментарий	Тип	Площадка
15	Концертный билет	(12.5; 100)	10.10.2026, 12:43:57	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 11)
18	Концертный билет	(12.5; 100)	10.10.2026, 13:41:19	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 14)
19	Концертный билет	(12.5; 100)	10.10.2026, 13:50:38	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 15)
20	Концертный билет	(12.5; 100)	17.10.2026, 15:30:46	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 16)
10	Концертный билет	(12.5; 100)	09.10.2026, 23:44:35	2500	Первый ряд	USUAL	Главный стадион, STADIUM, мест: 50000 (id 8)
11	Концертный билет	(12.5; 100)	09.10.2026, 23:44:56	2500	Первый ряд	USUAL	Главный стадион, STADIUM, мест: 50000 (id 9)
12	Концертный билет	(12.5; 100)	09.10.2026, 23:47:48	2500	Первый ряд	USUAL	Главный стадион, STADIUM, мест: 50000 (id 10)
5	Концерт 3	(3; 10)	09.10.2026, 22:32:38	350	c3	USUAL	Бар 3, BAR, мест: 10 (id 4)
4	Концерт 2	(2; 10)	09.10.2026, 22:32:38	200	c2	USUAL	Бар 2, BAR, мест: 10 (id 3)
7	Концерт 1	(1; 10)	09.10.2026, 22:36:50	200	c1	VIP	Бар 1, BAR, мест: 10 (id 5)
8	Концерт 1	(1; 10)	09.10.2026, 22:40:24	200	c1	VIP	Бар 1, BAR, мест: 10 (id 6)

Назад Страница 1 из 1, всего 11 Вперёд

▸ Полученный ответ

Рис 1. Список билетов с фильтрацией, сортировкой и постраничным выводом



Продать VIP-копию билета

id билета 10 id человека 1 Выполнить

Выполнено

id	11
ticketId	21
personId	1
createdAt	2026-10-24T09:06:14.736Z

▸ Полученный ответ

Рис 2. Продажа VIP-копии билета через Booking Service



Список

Фильтры (все условия через И)

+ фильтр

Сортировка (по порядку приоритета)

+ сортировка

Размер страницы 20 Показать

id	id билета	id человека	Создано
6	15	2	10.10.2026, 12:43:57
9	19	1	10.10.2026, 13:50:38
10	20	1	17.10.2026, 15:30:46
11	21	1	24.10.2026, 12:06:14

Назад Страница 1 из 1, всего 4 Вперёд

▸ Полученный ответ

Рис 3. Список бронирований

Swagger UI

https://se.ifmo.ru/~s412939/

В спецификациях указаны публичные https-адреса сервисов, поэтому операции можно выполнять прямо из Swagger UI.

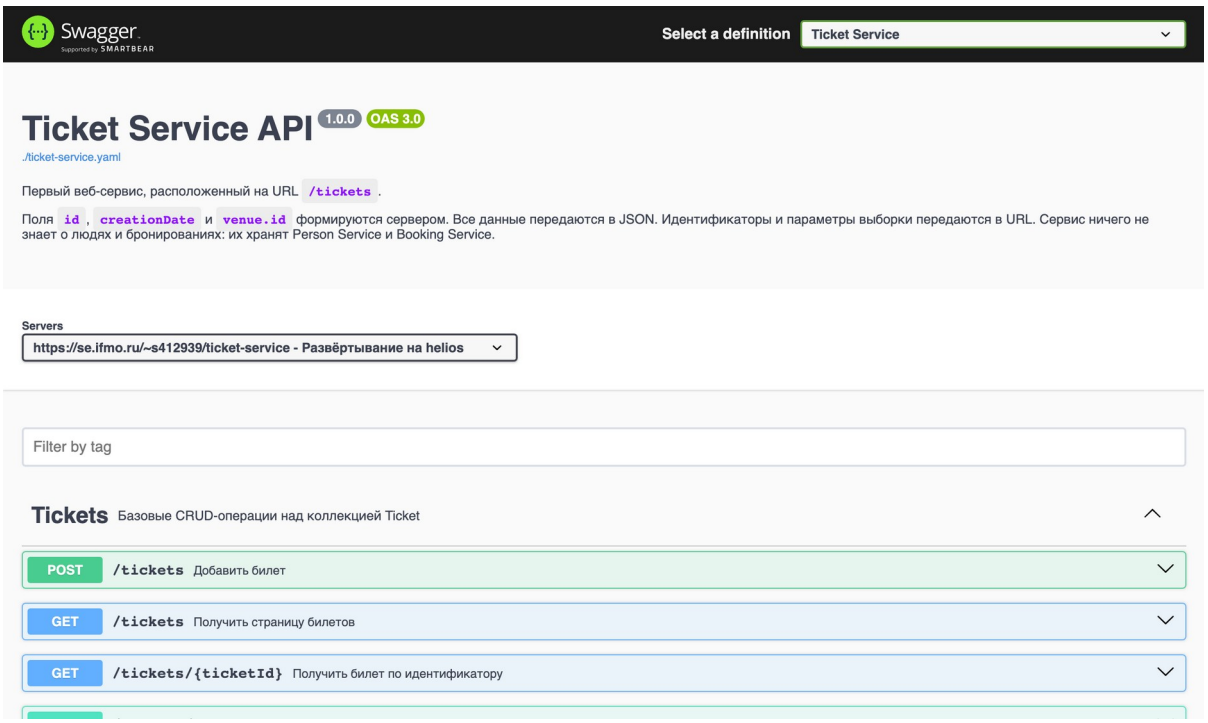


Рис 4. Swagger UI для Ticket Service

Выводы

Реализованы три сервиса на Spring MVC REST и клиент на React. Генерация из OpenAPI связывает контроллеры, модели и REST-клиенты с исходными контрактами. Запросы к PostgreSQL выполняют фильтрацию, сортировку и постраничную выборку. Ошибки имеют единый формат и русские и английские переводы. Обработка валидации возвращает первую ошибку. JSON Merge Patch сохраняет различие между отсутствующим полем и явным null. Для развёртывания подготовлены WAR-архивы, отдельные экземпляры Tomcat и HTTPS с самоподписанным сертификатом.